

Part 2

```
Hash functions operate on bytes, so the password must first be encoded.

>
import hashlib

password = "Cyber123!"

password_bytes = password.encode()

password_hash = hashlib.sha256(
    password_bytes
).hexdigest()

print("Password hash:")
print(password_hash)

[6] ✓ 0.0s

... Password hash:
34bf5eb46102238cd554f7a41353c5278073442ea98f7c30db33aaa4621f99e2
```

1. Yes it produces the same hash everytime
2. It is a 64 character string
3. Just by looking at the hash and not decoding it, no I cannot determine the original password

```
1 import hashlib
2
3 def hash_password(password: str) -> str:
4     """Hashes a plaintext password using SHA-256 and returns its hex digest."""
5     return hashlib.sha256(password.encode()).hexdigest()
6
7 def verify_password(password: str, stored_hash: str) -> bool:
8     """Verifies an entered password against a stored SHA-256 hash."""
9     return hash_password(password) == stored_hash
10
11 test_password = "Cyber123!"
12 hashed = hash_password(test_password)
13
14 print("Hashed Password:", hashed)
15 print("Correct Password Match:", verify_password("Cyber123!", hashed))
16 print("Incorrect Password Match:", verify_password("WrongPass!", hashed))

...

Problems 1 Output Debug Console Terminal Ports Jupyter

PS C:\Users\rkory\OneDrive\Desktop\FSCT8561-Labs-Ryan> & C:\Users\rkory\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\rkory\OneDrive\Desktop\FSCT8561-Labs-Ryan\Lab_3\hash.py
Hashed Password: 34bf5eb46102238cd554f7a41353c5278073442ea98f7c30db33aaa4621f99e2
Correct Password Match: True
Incorrect Password Match: False
PS C:\Users\rkory\OneDrive\Desktop\FSCT8561-Labs-Ryan> 
```

Part 3,4,5,6,7

```
--- PARTS 3 & 7: User Record Created ---  
User: alice  
Password Hash: 34bf5eb46102238cd554f7a41353c5278073442ea98f7c30db33aaa4621f99e2  
Static TOTP Secret: BKNA27SYIPNCDBVZ23XB4VP7KIT7V3AA  
-----  
--- PART 6: Testing OTP Verification ---  
Current Valid OTP: 585156  
1. Test Current OTP (585156): True  
2. Test Incorrect OTP ('000000'): False  
  
Waiting 35 seconds for the OTP to expire...  
3. Test Expired OTP (585156): False
```

```
import hashlib  
import time  
import pyotp  
  
# PART 2: Functions  
def hash_password(password: str) -> str:  
    return hashlib.sha256(password.encode()).hexdigest()  
  
def verify_password(password: str, stored_hash: str) -> bool:  
    return hash_password(password) == stored_hash  
  
# PART 6: OTP Verification Function  
def verify_otp(secret: str, otp: str) -> bool:  
    totp = pyotp.TOTP(secret)  
    return totp.verify(otp)  
  
# PARTS 3 & 7: Setup User Database with a Fixed Secret  
alice_secret = pyotp.random_base32()  
  
users = {  
    "alice": {  
        "password_hash": hash_password("Cyber123!"),  
        "totp_secret": alice_secret # Secret is saved into the user record  
    }  
}  
  
print("--- PARTS 3 & 7: User Record Created ---")  
print("User: alice")
```

```

print("Password Hash:", users["alice"]["password_hash"])
print("Static TOTP Secret:", users["alice"]["totp_secret"])
print("-" * 45)

# PARTS 5 & 6: Test OTP Generation & Verification

# Retrieve the secret directly from the user dictionary
user_secret = users["alice"]["totp_secret"]

# Generate current 6-digit code from that stored secret
totp = pyotp.TOTP(user_secret)
current_otp = totp.now()

print("--- PART 6: Testing OTP Verification ---")
print("Current Valid OTP:", current_otp)

# Test 1: Valid OTP
is_valid = verify_otp(user_secret, current_otp)
print(f"1. Test Current OTP ({current_otp}): {is_valid}")

# Test 2: Invalid 6-Digit Code
is_invalid = verify_otp(user_secret, "000000")
print(f"2. Test Incorrect OTP ('000000'): {is_invalid}")

# Test 3: Expired OTP
print("\nWaiting 35 seconds for the OTP to expire...")
time.sleep(35) # Pauses execution so the 30-second TOTP window rolls over

is_expired = verify_otp(user_secret, current_otp)
print(f"3. Test Expired OTP ({current_otp}): {is_expired}")

```

For the last part, I used the `time.sleep()` function in python to test the 30 second window

Part 5 Observations

No, `pyotp.random_base32()` does not generate the same value every time

Part 6 observations

1. The current OTP Validates to True when tested immediately within the 30 second window
2. Any code differing from the current time window's code will return as False

3. An expired OTP that attempts to reuse a code after the 30 second window expires returns as False

Part 9, 10, 11, 12, 13

Test 1 successful login

<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_server.py Server is waiting for a connection... Alice's current test OTP: 828625 (27s left in current window) Connected by: ('127.0.0.1', 63011) Received: AUTH alice Cyber123! Password verified for: alice Valid OTP to enter: 828625 (19s remaining) Received: OTP 828625 Authentication successful for: alice Server closed PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3></pre>	<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_client.py Enter your username: alice Enter your password: Server: OTP_REQUIRED Enter your 6-digit OTP: 828625 Server: ACCESS_GRANTED Access Granted! Login successful. Disconnected PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3></pre>
--	--

Test 2 invalid password

<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_server.py Server is waiting for a connection... Alice's current test OTP: 294066 (12s left in current window) Connected by: ('127.0.0.1', 63690) Received: AUTH alice password123 Auth failed for: alice Client disconnected unexpectedly Server closed PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> </pre>	<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_client.py Enter your username: alice Enter your password: Server: ACCESS_DENIED Access Denied! Invalid username or password. Disconnected PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3></pre>
--	--

Test 3 invalid OTP

<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_server.py Server is waiting for a connection... Alice's current test OTP: 805130 (26s left in current window) Connected by: ('127.0.0.1', 61044) Received: AUTH alice Cyber123! Password verified for: alice Valid OTP to enter: 805130 (17s remaining) Received: OTP 000000 Invalid OTP for: alice Client disconnected unexpectedly Server closed PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> </pre>	<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_client.py Enter your username: alice Enter your password: Server: OTP_REQUIRED Enter your 6-digit OTP: 000000 Server: ACCESS_DENIED Access Denied! Disconnected PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3></pre>
---	--

Test 4 expired OTP

<pre>PS C:\Users\Ryan\OneDrive\Desktop\FSC8561-Labs-Ryan-Lee\Lab 3> python .\auth_server.py Server is waiting for a connection... Alice's current test OTP: 461833 (16s left in current window) Connected by: ('127.0.0.1', 30680) Received: AUTH alice Cyber123! Password verified for: alice Valid OTP to enter: 461833 (7s remaining) Received: OTP 461833 Invalid OTP for: alice Client disconnected unexpectedly Server closed PS C:\Users\Ryan\OneDrive\Desktop\FSC8561-Labs-Ryan-Lee\Lab 3> </pre>	<pre>PS C:\Users\Ryan\OneDrive\Desktop\FSC8561-Labs-Ryan-Lee\Lab 3> python .\auth_client.py Enter your username: alice Enter your password: Server: OTP_REQUIRED Enter your 6-digit OTP: 461833 Server: ACCESS_DENIED Access Denied! Disconnected PS C:\Users\Ryan\OneDrive\Desktop\FSC8561-Labs-Ryan-Lee\Lab 3></pre>
--	--

Test 5 non existent username

<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_server.py Server is waiting for a connection... Alice's current test OTP: 210156 (5s left in current window) Connected by: ('127.0.0.1', 65091) Received: AUTH bob Cyber123! Auth failed for: bob Client disconnected unexpectedly Server closed PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> </pre>	<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_client.py Enter your username: bob Enter your password: Server: ACCESS_DENIED Access Denied! Invalid username or password. Disconnected PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3></pre>
---	--

Test 6 OTP before password

<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_server.py Server is waiting for a connection... Alice's current test OTP: 170572 (9s left in current window) Connected by: ('127.0.0.1', 53646) Received: OTP 123456 OTP rejected: Password not verified first Client disconnected unexpectedly Server closed PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> </pre>	<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_client.py Enter your username: alice Enter your password: Server: ACCESS_DENIED Access Denied! Invalid username or password. Disconnected PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3></pre>
--	--

For this test I temporarily changed the auth_message in the client

```
#auth_message = "AUTH|" + username + "|" + password
auth_message = "OTP|123456"

client_socket.send(
    auth_message.encode()
)
```

Test 7 malformed auth message

<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_server.py Server is waiting for a connection... Alice's current test OTP: 475997 (8s left in current window) Connected by: ('127.0.0.1', 58498) Received: AUTH alice Client disconnected unexpectedly Server closed PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> </pre>	<pre>PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3> python .\auth_client.py Enter your username: alice Enter your password: Server: ERROR Malformed AUTH message Disconnected PS C:\Users\rkory\OneDrive\Desktop\FSC8561-Labs-Ryan\Lab 3></pre>
--	--

For this test I temporarily changed the auth_message in the client

```
# auth_message = "AUTH|" + username + "|" + password
#test 6
# auth_message = "OTP|123456"
#test 7
auth_message = "AUTH|alice"

client_socket.send(
    auth_message.encode()
)
```

Test 8

Because nothing is implemented after repeated failed attempts, nothing happens to the server other than the error message after every attempt. Having unlimited attempts is bad because we are then prone to brute force attacks as the user can enter in the password an unlimited amount of times. The OTP can also be brute forced with an unlimited amount of attempts because there

are only 1 million possible combinations which can be brute forced pretty easily by a computer. Denial of service will also happen through resource exhaustion if there is an unlimited number of attempts because persistent TCP sockets will force the server to keep connections open and repeatedly perform CPU intensive hashing and memory operations.

Part 14

1.

TOTP generates a code from a shared secret and the current time. The authenticator app and server need to use the same time so they independent calculate the same code

2.

The server normally rejects the code because it corresponds to an earlier time interval and is no longer considered valid

3.

Device clocks can differ slightly because of network delays or imperfect time synchronization. Servers may then accept a code from a small neighbouring time window to avoid rejecting legitimate users.

4.

It increases the attack window because more different codes are accepted at a given moment. This makes brute force or guessing attacks somewhat easier, so the tolerance should be kept small.

Part 15

1.

If a database is breached, plaintext storage immediately exposes raw credentials. Most people use the same password across multiple services, a breach on one server compromises the user's accounts across other platforms.

2.

Hashing uses a one-way cryptographic function to transform a plaintext password into a fixed-length string. The server verifies user identity by hashing incoming login credentials and comparing the resulting digest against the stored digest, making sure raw passwords are never stored.

3.

SHA-256 is extremely fast and with modern GPUs, they would be able to execute billions of hash calculations per second in brute force attack. The passwords would be cracked in a few hours or even minutes.

4.

TOTP introduces a second authentication factor based on "something you have." Even if an attacker steals, keylogs, or phishes a static password, they cannot complete authentication without physical access to the seed-synced device generating the time-based tokens.

5.

TOTP codes are bound to specific 30-second time windows. Once the time step advances, the server invalidates that specific token, preventing an attacker from reusing an intercepted or eavesdropped code in a replay attack. Our server doesn't track used codes, so it isn't fully preventable.

6.

TOTP algorithms derive counter values using the UNIX epoch time. If the user and server clocks drift beyond the accepted tolerance window, they calculate different time steps, producing mismatched 6-digit codes and causing legitimate login attempts to fail.

7.

If an attacker obtains the Base32 TOTP seed, they can import it into their own authenticator application. This allows them to generate valid 6-digit codes concurrently with the legitimate user, completely bypassing the second authentication factor.

8.

Without rate limits or lockout thresholds, attackers can execute automated online brute-force scripts against passwords or the 1 million possible 6-digit TOTP combinations within an active window.

9.

No. `getpass()` only prevents local "shoulder surfing" by suppressing terminal character echoing while the user types. Once submitted, the client sends the raw password string across the network in cleartext unless underlying socket encryption (such as TLS/SSL) is implemented.

10.

- Wrapping TCP sockets with Transport Layer Security (TLS/SSL).
- Migrating from plain SHA-256 to salted, memory-hard hashing algorithms like Argon2id or bcrypt.
- Encrypting TOTP secrets at rest in the database using a hardware security module or key management service
- Implementing persistent rate limiting by IP/username with exponential backoff or temporary lockouts.
- Replacing single-request socket sessions with cryptographically signed, expiring session tokens

Part 16 Security Analysis

In my implementation, `auth_server.py` stores the password as a SHA-256 hash in a dictionary, so the plaintext "Cyber123!" never sits in the user database. When a client sends `AUTH|username|password`, the server hashes what it received and compares it to the stored digest. Adding PyOTP changed the login into a two-step process. The server only replies `OTP_REQUIRED` if the password matches, and it ignores any OTP message until `password_verified` is `True`. This helps against credential stuffing and password reuse, because a stolen password alone is not enough to get in. My tests backed this up: a wrong password, a fake username like bob, and an OTP sent before the password were all rejected without crashing the server.

That said, my system has some real weaknesses. The biggest one is the raw TCP socket. Even though `getpass()` hides the password on screen, it still travels across the network in cleartext, so anyone sniffing the traffic could read it. Plain unsalted SHA-256 is also a problem, since it is fast and an attacker with the hashes could crack them with a GPU or rainbow tables. The TOTP secret is hardcoded in the server file too, so if that file leaks, someone could generate valid codes on their own. I also used `valid_window=1`, which accepts neighbouring 30-second codes for clock drift, but that does widen the window an attacker has. On top of that, the server prints the current OTP in its logs, which is fine for testing but would be a leak in real life.

Repeated attempts were the last big issue. At first, the server allowed unlimited tries, and a wrong OTP did not reset `password_verified`, so someone could keep guessing from the 1,000,000 possible codes on the same connection. In Part 17 I fixed this by adding a failed-attempt counter. Now a wrong password or wrong OTP counts as one failure, a wrong OTP sends the user back to the password step, and after 3 failures the server blocks that connection, even if the credentials are correct. The limitation is that the counter only lives in memory for one connection, so an attacker can just reconnect and get 3 fresh tries. For a real deployment I would add TLS, switch to a salted, memory-hard hash like Argon2id or bcrypt, store the TOTP secrets encrypted, and track failed attempts per account and IP with temporary lockouts.

Part 17 Challenge

```
PS C:\Users\Ryan\OneDrive\Desktop\FSC78561-Labs-Ryan-Lee\Lab 3\Challenge> python .\auth_server.py
Server is waiting for a connection...
Alice's current test OTP: 077760 (21s left in current window)
Connected by: ('127.0.0.1', 2889)
Received: AUTH|alice|Cyber123!
Password verified for: alice
Valid OTP to enter: 077760 (12s remaining)
Received: OTP|000000
Invalid OTP for: alice
Failed attempts: 1 / 3
Received: AUTH|alice|password
Auth failed for: alice
Failed attempts: 2 / 3
Received: AUTH|bob|Cyber123!
Auth failed for: bob
Failed attempts: 3 / 3
Too many failed attempts. Authentication blocked.
Received: AUTH|alice|Cyber123!
Blocked: too many failed attempts, request denied
Received: AUTH|alice|Cyber123!
Blocked: too many failed attempts, request denied
Client disconnected unexpectedly
Server closed
PS C:\Users\Ryan\OneDrive\Desktop\FSC78561-Labs-Ryan-Lee\Lab 3\Challenge>

PS C:\Users\Ryan\OneDrive\Desktop\FSC78561-Labs-Ryan-Lee\Lab 3\Challenge> python .\auth_client.py
Enter your username: alice
Enter your password:
Server: OTP_REQUIRED
Enter your 6-digit OTP: 000000
Server: ACCESS_DENIED
Access Denied!
Try again? (y/n): y
Enter your username: alice
Enter your password:
Server: ACCESS_DENIED
Access Denied! Invalid username or password.
Try again? (y/n): y
Enter your username: bob
Enter your password:
Server: ERROR|Too many failed attempts. Authentication blocked.
Authentication blocked.
Try again? (y/n): y
Enter your username: alice
Enter your password:
Server: ERROR|Too many failed attempts. Authentication blocked.
Authentication blocked.
Try again? (y/n): y
Enter your username: alice
Enter your password:
Server: ERROR|Too many failed attempts. Authentication blocked.
Authentication blocked.
Try again? (y/n): n
Disconnected
PS C:\Users\Ryan\OneDrive\Desktop\FSC78561-Labs-Ryan-Lee\Lab 3\Challenge>
```

I implemented a system where after 3 failed attempts, it locks the user out, even if they enter the correct credentials after the failed attempts.

Challenge Questions

1.

In our basic lab script, `failed_attempts` is held in local memory for that single TCP socket connection. If an attacker simply disconnects and opens a new connection, the counter resets back to zero. In production, this counter should be stored in a centralized, fast key-value datastore indexed by username and client IP address.

2.

No, they should be handled differently because they represent different threat levels. Password failures usually point to forgotten passwords, credential stuffing, or basic brute-force attempts. These should trigger IP-level rate limiting or soft lockouts to prevent automated guessing. OTP failures occur after the static password has already been correctly verified. A failed OTP means either the user's clock is out of sync or an attacker has already compromised the password and is trying to guess the 6-digit MFA token. Because the primary credential is already exposed, OTP failures should trigger stricter, immediate lockouts, force password resets, and alert the user.

3.

If an account permanently locks after 3 failed attempts, an attacker can exploit the feature to lock legitimate users out on purpose. An attacker could run a script that sends 3 incorrect

passwords for a targeted user (like admin or an executive email), forcing the server to lock the account. This turns a security control intended for protection into an easy Account Lockout DoS attack vector.

4.

Temporary Soft Lockouts & Exponential Backoff: Introduce escalating delay timers (like 1 second, 5 seconds, 30 seconds) between failed attempts, or lock the account for a short duration (ex: 15 minutes) rather than indefinitely.

IP-Based Throttling & CAPTCHAs: Rate limit requests from specific IP addresses and present a CAPTCHA challenge after 2 failed attempts to block automated bot traffic without locking the account globally.

Send an automated email notification to the account holder warning them of failed login attempts, providing a secure, signed link to unlock the account or trigger an emergency password reset.